Carnet métacognitif — usage de l'IA

Projet 1 : Messagerie temps réel (B2B) — 5IDEV · 2026-2027

Nom & prénom	van der Veen Georgé
Groupe / app	Coisne Valentin & van der Veen Georgé — **Canopée**, messagerie en temps réel (Django + React)
Mon rôle	Backend (API Django REST, WebSockets avec Django Channels), tests, documentation (devlog, rapports)

Ce carnet est individuel : il décrit **mon** usage de l'IA. Valentin décrit le sien dans son propre carnet.

J'ai demandé à Claude de rédiger une première version de ce carnet à partir de notre conversation, puisqu'elle contient tout l'historique de mon utilisation. Je l'ai ensuite vérifiée, corrigée et complétée.

IMPORTANT (écrit par moi-même) : j'écris ici un résumé qui exprime mon ressenti. Comme dit plus loin, pour le back, j'ai utilisé l'IA seulement comme un guide la plupart du temps, en essayant de bien comprendre ce que je faisais (il m'est arrivé quelques fois de faiblir et de demander la solution). J'ai ainsi bien compris comment marchait le workflow Git, les requêtes HTTP, comment implémenter des routes dans Django, comment on fetch dans le front, et comment tout fonctionnait dans sa globalité. Néanmoins, l'IA m'a quand même guidé en me répondant sur la manière de développer, que ce soit en donnant des idées d'architecture ou l'ordre dans lequel écrire le code. Je pense que j'aurais du mal à refaire certaines choses moi-même, c'est pourquoi j'ai écrit (toujours avec son aide) un devlog qui décrit les étapes faites, ainsi que des cheatsheets, pour pouvoir tout refaire moi-même un jour.

Je trouve que c'est un bon outil auquel il est tentant de tout laisser faire (par exemple, le rapport et ce carnet ont en grande partie été écrits par lui), et c'est important pour être compétitif dans le monde du travail. Mais je ne sais pas si, dans le cadre d'un cours, c'est toujours utile de lui laisser autant de place juste pour aller plus vite. Pour les prochains projets, je tâcherai de me limiter encore plus afin de pouvoir reproduire ce que je fais plus simplement. Même si je pense qu'à force de lui demander la même chose on finit par apprendre, je pense que ça n'ira jamais aussi vite que la recherche par soi-même.

Partie 1 — Déclaration d'usage

1. Mon setup IA en détail

☒ **Outils et abonnements.** Un seul outil : **Claude** (Anthropic), dans l'application Claude, avec l'abonnement payant le moins cher. Je l'ai utilisé de deux façons :

- en **chat** classique, pour une question ponctuelle ;
- en **mode agent** : Claude travaille dans un espace Linux dans le cloud, où il peut cloner notre dépôt GitHub, lire le code, lancer les tests, démarrer l'application en local, faire des captures d'écran, puis m'envoyer des fichiers.

☒ **Modèles.** Claude Opus 5.5, avec un niveau de réflexion élevé, pendant tout le développement : comme je l'utilisais surtout pour me guider et m'expliquer les fonctions utiles, je voulais les explications les plus fiables possible.

☒ **Instructions personnalisées.** Mes préférences globales, actives dans toutes mes conversations, sont faites pour apprendre et pas seulement recevoir du code :

> *« Quand je demande d'apprendre un sujet : procède une étape à la fois et attends ma réponse avant de continuer. Pour le code, ne donne jamais la solution complète d'emblée — donne la structure et des indices gradués, et laisse-moi essayer. Si je bloque, donne un indice, pas la réponse, sauf si je demande explicitement « donne-moi la solution ». »*

J'ai ajouté des consignes en cours de route, quand quelque chose ne me convenait pas :

- « attends, tu vas trop vite » : une étape à la fois ;
- résumer la documentation dont j'ai besoin au lieu de m'envoyer des liens ;
- « ne me donne jamais de commit, donne-moi le fichier directement » : c'est moi qui fais les commits et les Pull Requests ;

- quand Claude écrit entièrement du code (par exemple une partie du frontend, qui n'était pas mon rôle), le dire dans le commit (« Frontend written by Claude », `Co-Authored-By`, « made by claude »).

☒ **Skills et workflows.** Pas de skill personnalisé. J'ai utilisé des capacités intégrées : lecture des PDF (consignes, grille d'évaluation), création du graphique des commits, compilation des rapports en PDF (Typst). Mon vrai « workflow », c'est le **devlog** (`docs/DEVLOG.md`) : après chaque fonctionnalité, on y réexplique le code et les erreurs rencontrées.

☒ **MCP / intégrations.** Accès en lecture à notre dépôt GitHub, recherche web, et recherche dans mes anciennes conversations (utilisée pour ce carnet).

☒ **Autres éléments.** Je n'ai configuré ni hooks ni sous-agents. Le garde-fou principal est une règle : Claude ne touche jamais à Git sur le dépôt (voir question 5).

2. Niveau d'usage global

Niveau 3 — coproduction. Pour être honnête, il faut séparer deux parties :

- **le backend**, mon rôle : je l'ai écrit moi-même, avec Claude comme tuteur (structure et indices, puis correction de mon code). Parfois, bloqué, je lui ai demandé la solution (« corrige », « donne-moi ») ;
- **le reste** : Claude a écrit une part importante, que j'ai dirigée et vérifiée : une partie du frontend, 35 des 45 tests, la configuration de ruff, une grande partie de la documentation, et la mise en forme du rapport avec ses schémas et ses captures (détails en question 5).

3. J'ai utilisé l'IA pour...

☒ comprendre les consignes ou une notion du cours
☒ déboguer / comprendre une erreur
☒ générer du code (une partie du frontend, pas le backend)
☒ configurer le CI (ruff) ; le CD a été fait par Valentin
☒ générer des éléments visuels (schémas, graphique, captures)
☒ chercher des idées (fonctionnalités, architecture…)
☒ élaborer un plan / structurer (rapport, modèle de données)
☒ écrire des tests
☒ rédiger ou reformuler du texte (rapport, README, devlog)
☒ autre : Git (branches, conflits), descriptions de PR

Partie 2 — Démarche & vérification

4. Démarche suivie

J'ai utilisé l'IA **tout au long du projet**, à chaque fonctionnalité backend : inscription, JWT, modèles, API, tests et CI (septembre et début octobre) ; WebSockets (octobre) ; ajout de participants, durée des tokens, conversations privées et groupes, renommage (fin octobre et début novembre) ; puis audit final et rapport.

Ma façon de demander était presque toujours la même :

1. décrire la fonctionnalité et demander « aide-moi, mais donne-moi de la doc » ;
2. recevoir la structure et des indices ;
3. écrire le code, le coller dans le chat ;
4. recevoir une correction expliquée ;
5. itérer jusqu'à ce que ça marche ;
6. demander la correction du devlog écrit en parallèle.

Exemple réussi : l'ajout d'un participant (`POST /api/conversations/<id>/participants/`). J'ai écrit la vue en plusieurs essais. Claude ne m'a pas donné la solution, il m'a montré mes erreurs :

- `get_object_or_404(User, id=username)` : filtrer sur un champ qui ne correspond pas à la valeur qu'on a ;
- `Conversation.participants` (la classe) au lieu de `conversation.participants` (l'objet).

J'ai aussi demandé qu'il m'explique en détail `get_object_or_404` dans le devlog : je sais maintenant pourquoi on y met le filtre `participants=request.user`, et pourquoi ça renvoie 404 et pas 403.

Exemple moins réussi : les conversations privées et de groupe. L'idée vient de moi : ajouter quelqu'un à une conversation privée crée un nouveau groupe. Mais dans `perform_create`, mes conditions étaient fausses plusieurs fois de suite. J'avais oublié que `others` m'excluait, donc `len(others) < 2` refusait toutes les conversations privées. J'ai aussi laissé une ancienne ligne dans la vue. Après plusieurs essais, j'ai demandé « corrige ». J'ai compris la correction après coup, mais j'ai moins appris que si j'avais trouvé moi-même.

5. Ce que j'ai délégué à un agent

- **L'interface des groupes et du renommage** dans `ChatPage.jsx`. C'est la partie de Valentin, et j'ai choisi de la confier à Claude plutôt que de l'écrire moi-même. Consigne : « dis-moi les gros changements que tu fais ». Les commits le disent (« Frontend written by Claude »).
- **Les tests et la qualité** :
  - configuration de ruff et ajout dans la CI ;
  - 35 nouveaux tests : messages, conversations, groupes, WebSocket, inscription.
- **La documentation** : réécriture du devlog, du README et du cahier des charges, et rapports d'avancement.
- **Un audit du dépôt** avec la grille d'évaluation, qui a donné la liste des priorités dans `docs/TODO.md`.

Garde-fous :

- pas de commit ni de push par l'agent ;
- des fichiers complets que je relis avant de les intégrer ;
- toujours une Pull Request, avec la CI et l'avis de mon binôme ;
- chaque fonctionnalité testée à la main avec deux navigateurs ;
- les tests relancés avant chaque livraison de code.

6. Vérifications réalisées

- **Backend** : je vérifiais chaque ligne, puisque c'est moi qui l'écrivais, puis les tests automatiques (lancés par la CI à chaque Pull Request) me permettaient de voir si mon code cassait quelque chose.
- **Frontend** : je testais à la main, avec deux navigateurs connectés à deux comptes pour le temps réel, et j'essayais de comprendre le code React fourni par Claude, surtout au début.
- **Binôme** : Valentin relisait les Pull Requests.

Ces vérifications ont détecté des problèmes. Par exemple :

- plusieurs fois, en me donnant des fichiers pour le frontend, Claude a aussi modifié le backend alors que mon code fonctionnait déjà : j'ai refusé ces changements ;
- le code recopié dans le devlog ne correspondait plus à celui de GitHub : je lui ai demandé de tout revérifier contre le dépôt ;
- quand l'ajout d'un participant renvoyait une erreur 404, Claude pensait d'abord que je n'avais pas le bon code backend. Je lui ai montré mon code, ce qui a permis de trouver la vraie cause : un ancien serveur tournait encore ;
- en testant à la main, j'ai vu que créer une conversation avec quelqu'un ouvrait parfois un ancien groupe au lieu de notre conversation privée (voir question 8).

Partie 3 — Regard critique

7. Ce que l'IA m'a apporté

Surtout de l'**apprentissage**. Je découvrais Django, Django REST Framework et Django Channels. Avoir quelqu'un qui explique **mon** erreur, sur **mon** code, au moment où je la fais, m'a fait progresser beaucoup plus vite qu'avec la documentation seule. Le devlog qui en est sorti (plus de 1 200 lignes) est devenu mon cours personnel : j'y retourne quand je dois refaire une route.

Ensuite, le **débogage**, où j'aurais perdu des heures seul : par exemple les déconnexions après 5 minutes (durée de vie du token par défaut), ou un ancien serveur qui tournait encore et répondait « URL introuvable ».

Enfin, le **temps** : déléguer une partie du frontend, des tests et la mise en page du rapport nous a permis de terminer des fonctionnalités optionnelles (groupes, renommage) avant la date limite.

8. Ses limites et erreurs

- **Trop vite, trop d'un coup.** Au début, il donnait plusieurs étapes et beaucoup de code à la fois : j'ai dû le ralentir (voir mes consignes, question 1).
- **Il proposait plus que nécessaire.** Le premier cahier des charges et le devlog prévoyaient Redis et PostgreSQL. Nous avons décidé que SQLite et la couche en mémoire suffisaient pour notre nombre d'utilisateurs, et il a fallu corriger la documentation qui affirmait le contraire.
- **La documentation dérive.** Comme il écrit vite et beaucoup, le devlog et le README ont fini par ne plus correspondre au code. Il faut les revérifier contre le dépôt.
- **Il ne voit ni mon ordinateur ni la production.** Commit non poussé, ancien serveur encore lancé, branche supprimée sur GitHub, site en ligne : il ne pouvait pas le vérifier lui-même, c'est moi qui devais lui donner ces informations.
- **Il n'a pas anticipé un problème de données.** Quand on a ajouté `is_group` (faux par défaut), les anciens groupes sont devenus « privés » et une conversation privée ouvrait un ancien groupe. Ni lui ni moi ne l'avions prévu, d'où la migration de données `0003`.

9. Ce que j'ai appris (ou pas) en l'utilisant

Ce que je saurais refaire sans IA, avec le devlog et la documentation :

- un modèle Django avec une relation N-N et des migrations ;
- un serializer avec des champs `read_only` / `write_only` et `SlugRelatedField` ;
- une vue `ListCreateAPIView` ou `APIView` avec un contrôle d'accès par `get_object_or_404(..., participants=request.user)` ;
- la configuration JWT ;
- un consumer Channels simple et l'envoi d'événements avec `group_send` ;
- des tests DRF simples.

J'ai aussi approfondi des notions :

- la différence entre ASGI et WSGI ;
- le fonctionnement précis des requêtes HTTP.

Et Git : branches de fonctionnalité, Pull Requests, conflits, `git stash`, vérifier qu'un commit est poussé, merge plutôt que squash. C'est sans doute ce que j'ai le plus appris en pratique.

Ce que je ne saurais pas encore refaire seul :

- le frontend écrit par Claude : je comprends l'idée, mais je ne le réécrirais pas sans aide (les closures React avec `useRef` par exemple) ;
- le test WebSocket asynchrone.

Je sais lire et expliquer les tests, mais je n'en ai écrit qu'une partie.

Sans IA, j'aurais appris plus lentement et le projet aurait eu moins de fonctionnalités. J'aurais peut-être mieux retenu certaines erreurs en les cherchant seul plus longtemps. Il aurait aussi été difficile de mener un projet comme celui-ci sans elle, car elle m'a beaucoup aidé à savoir quoi faire à quel moment. Ce ne serait cependant pas impossible, notamment grâce au devlog que j'ai tenu.

10. Contribution personnelle

- **Le backend, c'est mon code.** Inscription, modèles, API des conversations et des messages, contrôle d'accès, WebSockets, ajout de participants, conversations privées et groupes, renommage : écrit par moi, avec l'IA comme tuteur.
- **Des décisions qui viennent de moi** :
  - ajouter quelqu'un à une conversation privée crée un nouveau groupe, sans l'historique ;
  - garder SQLite et la couche en mémoire au lieu de PostgreSQL et Redis ;
  - ce que l'IA écrit et ce que j'écris moi-même ;
  - la priorité des tâches avant la remise.

- **L'organisation de l'équipe** : branches, Pull Requests, merges vers `develop` et `main`, résolution de conflits, coordination avec Valentin.
- **La vérification** (question 6), et le contenu du rapport, que j'ai relu, corrigé et complété : les réponses aux questions sur notre projet viennent de nous.

Je peux expliquer chaque route du backend et défendre chaque choix du rapport.

Attestation. Je déclare que ce carnet reflète honnêtement mon usage de l'IA et que je suis en mesure d'expliquer et de défendre l'ensemble du travail remis.

Nom : van der Veen Georgé Date : 02/11/2026